

# Universidade do Algarve

## Programação Imperativa

Ficha de exercícios online nº4

site mooshak: <http://deei-mooshak.ualg.pt/~pi>

Para sua informação, deixamos aqui o aviso que vigora durante as provas de avaliação, nomeadamente nas festas e nos exames:

### Aviso Geral

- Use o computador **exclusivamente** para escrever, compilar e submeter programas C ao mooshak. **Tolera-se** a consulta dos acetatos das aulas teóricas arquivadas no disco. Qualquer outro uso do computador que não seja resolver os problemas propostos, **não é autorizado** durante a festa e **qualifica-se como fraude**.
- Qualquer uso de material indevido (telemóveis, *chats*, pdfs etc...) é sancionado com **reprovação imediata à UC de Programação Imperativa e será assinalada às autoridades académicas competentes**.
- Qualquer comportamento indevido, não autorizado ou fraude académica, etc... é **sancionado com reprovação imediata à UC de Programação Imperativa e será assinalada às autoridades académicas competentes**.
- A elegância e eficiência do código serão elementos de avaliação.  
Por exemplo, recorra à definição de funções sempre que justificado.
- Um exercício pode exigir uma determinada solução (por exemplo, “*não usar o tipo float*”, “*não usar ciclos*”, etc.). Uma solução aceite pelo mooshak mas que não respeita estas exigências será avaliada **para metade da sua cotação**.

### As regras do mooshak

- `gcc -Wall -lm ...`      A função `main` deverá sempre devolver `0`.
- Uma linha de input ou de output **termina sempre com \n**.
- input/output: Não há espaços a não ser os que estão explicitamente referidos no enunciado.
- **Deixe sempre uma linha em branco** no fim do ficheiro submetido.

## Exercício 1 (A)

Vamos implementar neste exercício um algoritmo histórico, foi formulado pelo próprio Euclides, 300 AC. este algoritmo descreve o cálculo do máximo divisor comum de dois inteiros.

Em particular, interessa-nos a versão recursiva.

$$\text{mdc}(a, b) = \begin{cases} a & \text{se } b = 0 \\ \text{mdc}(b, \text{mod}(a, b)) & \text{senão} \end{cases}$$

(onde *mod* é a função que devolve o resto da divisão do seu primeiro argumento com o seu segundo). Recorde-se que na linguagem C, esta função é o operador %.

Defina uma função recursiva *mdc*, cujo protótipo em C é:

```
int mdc(int a, int b);
```

que aceite dois inteiros e devolve um inteiro, que calcula o maior divisor comum com base no algoritmo acima descrito. Assim,  $\text{mdc}(36, 45) = 9$ .

**input:** Duas linhas com um argumento inteiro em cada uma delas.

Por exemplo:

```
36
45
```

**output:** Uma linha com o resultado inteiro da função, ou com a palavra “NO” em caso de qualquer anomalia.

Por exemplo (em resposta ao exemplo de input anterior):

```
9
```

□

## Exercício 2 (B)

Considere a função *tribonacci* sobre inteiros *naturais* na sua formulação seguinte:

$$\text{tribonacci}(n) = \begin{cases} 0 & \text{se } n = 0 \\ 1 & \text{se } n = 1 \\ 2 & \text{se } n = 2 \\ \text{tribonacci}(n-1) + \text{tribonacci}(n-2) + \text{tribonacci}(n-3) & \text{se } n > 2 \end{cases}$$

Defina a função recursiva em C.

**input:** Uma linha com um inteiro  $n$ . É garantido, nesta versão do exercício, que o inteiro  $n$  dado não ocasionará um `long long int` overflow e que o cálculo não será demorado.

Por exemplo:

7

**output:** Uma linha com o o resultado da aplicação da função *tribonacci* ao parâmetro efectivo  $n$  lido. Ou uma linha com a palavra “NO”, em caso de anomalia.

Por exemplo (em resposta ao exemplo de input anterior):

37

□

### Exercício 3 (C)

Este exercício debruça-se sobre a função de exponenciação inteira  $x^n$ . A definição natural e directa desta operação  $x^n = \underbrace{x \times x \times \dots \times x}_{n \text{ vezes}}$  sugere  $n - 1$  multiplicações. Mas uma definição alternativa simples, mas não tão trivialmente directa, permite realizar o mesmo cálculo minorando o número de multiplicações:

$$x^n = \begin{cases} 1 & \text{se } n = 0 \\ x & \text{se } n = 1 \\ x^{\lfloor \frac{n}{2} \rfloor} \times x^{\lfloor \frac{n}{2} \rfloor} & \text{se } n \text{ par} \\ x^{\lfloor \frac{n}{2} \rfloor} \times x^{\lfloor \frac{n}{2} \rfloor} \times x & \text{se } n \text{ ímpar} \end{cases}$$

Tal método de exponenciação é, as vezes, designada de **exponenciação rápida**. Proponha uma função exclusivamente recursiva em C que implemente esta definição. Vamos aqui assumir para simplificar que o valor do expoente é um inteiro positivo ou nulo (isto é: pertence a  $\mathbb{N}$ ).

**input:** Uma primeira linha com um número flutuante  $x$ . Uma segunda linha com um inteiro  $n$ .

Por exemplo:

2.5  
12

**output:** Uma linha com o valor  $x^n$  com uma precisão decimal de 6 décimas. Ou uma linha com a palavra “NO”, em caso de anomalia.

Por exemplo (em resposta ao exemplo de input anterior):

59604,644775

□

#### Exercício 4 (D)

Uma famosa sequência de números inteiros conhecida por **Números de Catalan** (do matemático belga Eugène Charles Catalan, 1814–1894) é definida por:

$$Catalan(n) = \begin{cases} 1 & \text{se } n = 0 \vee n = 1 \\ \sum_{(p,q) \text{ tais que } p+q=n-1} Catalan(p) * Catalan(q) & \text{se } n > 1. \end{cases}$$

Esta sequência está envolvida na contagem de variadíssimos fenómenos combinatórios (ver a página [wikipédia aqui](#)).

Escreva uma função `catalan` em `C` que receba um inteiro natural `n` e que calcule recursivamente o valor de `Catalan(n)`.

**input:** Uma linha com o valor inteiro `n`. Nesta versão do exercício é garantido que o resultado de `Catalan(n)` não ocasiona um overflow nos `long long int` nos valores para os quais está definido, nem demora um tempo significativo para o seu cálculo conforme a definição acima apresentada.

Por exemplo:

6

**output:** Uma linha com o valor de `Catalan(n)` ou uma linha com a palavra “NO”, em caso de anomalia.

Por exemplo (em resposta ao exemplo de input anterior):

132

□

**Exercício 5 (E)**

A fórmula de Wallis permite uma aproximação de  $\Pi$  definida nos termos seguintes:

$$\frac{\Pi}{2} = \frac{2 \times 2}{1 \times 3} \times \frac{4 \times 4}{3 \times 5} \times \frac{6 \times 6}{5 \times 7} \cdots$$

Proponha uma definição recursiva desta aproximação, na forma de uma função wallis que depende de um índice  $k \geq 1$ .

Se  $k = 1$ , a fórmula calculada, wallis(1), é  $2 \times \frac{2 \times 2}{1 \times 3}$ .

Se  $k = 2$ , a fórmula calculada, wallis(2), é  $2 \times \frac{2 \times 2}{1 \times 3} \times \frac{4 \times 4}{3 \times 5}$ .

Se  $k = 3$ , a fórmula calculada, wallis(3), é  $2 \times \frac{2 \times 2}{1 \times 3} \times \frac{4 \times 4}{3 \times 5} \times \frac{6 \times 6}{5 \times 7}$ , etc.

**input:** Uma linha com um valor inteiro  $k$ . Por exemplo:

5

**output:** uma valor flutuante de dupla precisão e com uma precisão de 12 decimais. Este valor é a aproximação de  $\pi$  pela fórmula de Wallis com  $k$  iterações. Ou uma linha com a palavra “NO”, em caso de anomalia.

Por exemplo (em resposta ao exemplo de input anterior):

3.002175954557

□

**Exercício 6 (F)**

Escreva uma função recursiva em C que recebe um inteiro  $n$  e um algarismo  $i$  e devolva o número de vezes que este algarismo  $i$  aparece em  $n$ .

**input:** Uma linha com o valor inteiro  $n$ . Uma segunda linha com o algarismo  $i$ .

*Por exemplo:*

1073741823

3

**output:** Uma linha com o número de vezes em que  $i$  aparece em  $n$ , ou uma linha com a palavra “NO”, em caso de anomalia. Por exemplo (em resposta ao exemplo de input anterior):

2

□

### Exercício 7 (G)

Vamos jogar um jogo simples que envolve dinheiro. Se eu lhe der uma soma inicial, digamos  $N$  euros, deverá devolver-me dinheiro desta soma conforme regras pre-estabelecidas que se aplicam até nenhuma se aplicar mais. Se conseguir, com essas regras ficar com exactamente 42 euros em mão, então ganhou o jogo e fica com esta soma.

Se assumirmos que tem  $m$  euros em mão, as regras são:

- (regra 1) se  $m$  for par, pode devolver-me  $\frac{m}{2}$  euros;
- (regra 2) se  $m$  for um múltiplo de 3 ou de 4 então pode multiplicar os dois últimos dígitos de  $m$  e devolver-me esta quantidade em euros;
- (regra 3) se  $m$  for um múltiplo de 5 então pode devolver-me exactamente 42 euros.

**input:** Uma linha com um inteiro  $N$ , a soma inicialmente proposta para o jogo. É garantido nesta versão do exercício que o valor  $N$  é razoável e não ocasiona tempos de resposta desmedidos.

Por exemplo:

250

**output:** De duas, uma:

- uma linha com um inteiro  $p$  que representa o número mínimo de passos (o número de regras) que usou para ganhar o jogo;
- ou uma linha com o texto *BAD LUCK*, caso não consiga ganhar.
- ou uma linha com a palavra *NO*, em caso de anomalia.

Por exemplo (em resposta ao exemplo de input anterior):

4

**Explicação:**

$$250 \xrightarrow{3} 208 \xrightarrow{1} 104 \xrightarrow{1} 52 \xrightarrow{2} 42$$

□



### Exercício 8 (H)

Observe o seguinte padrão constituído de espaços e de asteriscos.

```
*
* *
  *
* * * *
    *
  * *
    *
* * * * * * *
      *
    * *
      *
    * * * *
      *
        * *
        *
```

Descubra as regras de construção que o estrutura e escreva uma função recursiva que dado um parâmetro inteiro  $n$ , potência de 2 pertencente ao intervalo  $[1 \dots 100]$ , reproduza este padrão tal que a maior quantidade em linha de asteriscos seja  $n$ .

**input:** A entrada deste exercício consiste numa linha onde consta o inteiro  $n$ .

Por exemplo:

8

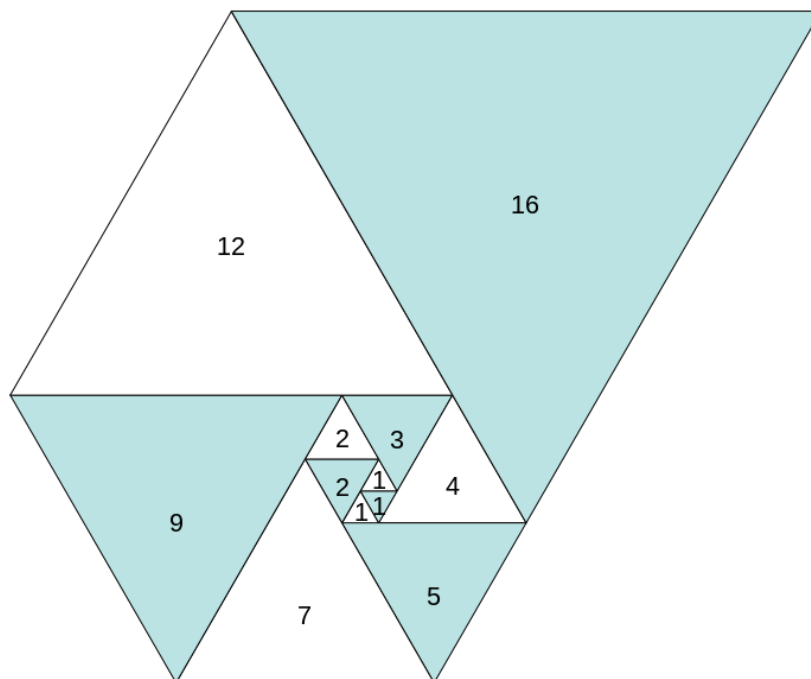
**output:** As linhas do determinado padrão. Ou a palavra *NO*, em caso de anomalia.

Por exemplo (em resposta ao exemplo de input anterior), o padrão dado acima.

□

### Exercício 9 (Problema I)

Considere a sequência ilustrada nesta imagem:



Reparemos que os três primeiros elementos desta sequência (que começa em 0!) são 1, 1 e 1. Tenha em atenção que o primeiro elemento é o elemento número 0. O seu desafio é calcular o  $n$ -ésimo elemento desta sequência.

**input:** Uma linha com um inteiro  $n$ . Garante-se que este valor é compatível com o tipo `int` no C.

Por exemplo:

9

**output:** uma linha com o  $n$ -ésimo elemento da sequência ou então a palavra `NO` se uma qualquer anomalia no processo for detectada.

Por exemplo (em resposta ao exemplo de input anterior):

9

□

### Exercício 10 (J)

Na sua história, a Índia teve o seu Leonardo de Fibonacci, mas com outro nome e com uma reflexão sobre outro ser vivo: os bovídeos. Dizia Narayana que se lhe dessem uma vaca, ele criaria uma manada da seguinte forma: Uma vaca produz um bezerro por ano. A partir do seu quarto ano, cada bezerro entra na vida adulta e produz um bezerro a cada ano. Quantas vacas tem ele após um  $x$  anos? **Este é o seu desafio!** Para perceber o fenómeno, olhemos para os primeiros 5 anos. No primeiro ano, só temos uma vaca. Mas ela produz um bezerro. No segundo ano, esta mesma vaca continua sendo o único bovídeo adulto da manada do Narayana mas gera um segundo bezerro. No terceiro ano, igual. No quarto ano, o primeiro bezerro entra na vida adulta e gera um bezerro, à semelhança da vaca sénior. São assim dois novos bezerros gerados, e duas vacas. No quinto ano, o segundo bezerro gerado também entra na vida adulta e gera também um bezerro. São 3 vacas. etc. Repare que a sequência começa **no ano 1 e com 1 vaca**. Resumidamente, em oito anos temos: 1, 1, 1, 2, 3, 4, 6, 9.

**input:** Uma linha, contendo um inteiro. É garantido que o valor introduzido no input é um valor inteiro. Por exemplo:

10

**output:** uma linha com o número de vacas da manada que corresponde ao ano introduzido, ou então a palavra NO se uma qualquer anomalia no processo for detectada. Por exemplo (em resposta ao exemplo de input anterior):

19

□